S3 Security

Explore how to secure Amazon S3 data using server-side encryption, configure CORS for cross-domain access, protect objects with MFA delete, analyze access logs for auditing, and create presigned URLs for controlled temporary access. This lesson equips you to manage and safeguard your S3 storage with AWS best practices and security features.

Security and compliance are top priorities for any modern application, and Amazon S3 provides robust encryption capabilities to help us protect our data at rest.

Types of server-side encryption in S3

AWS offers three main types of server-side encryption (SSE) for S3: SSE-S3, SSE-KMS, and SSE with customer-provided keys (SSE-C).

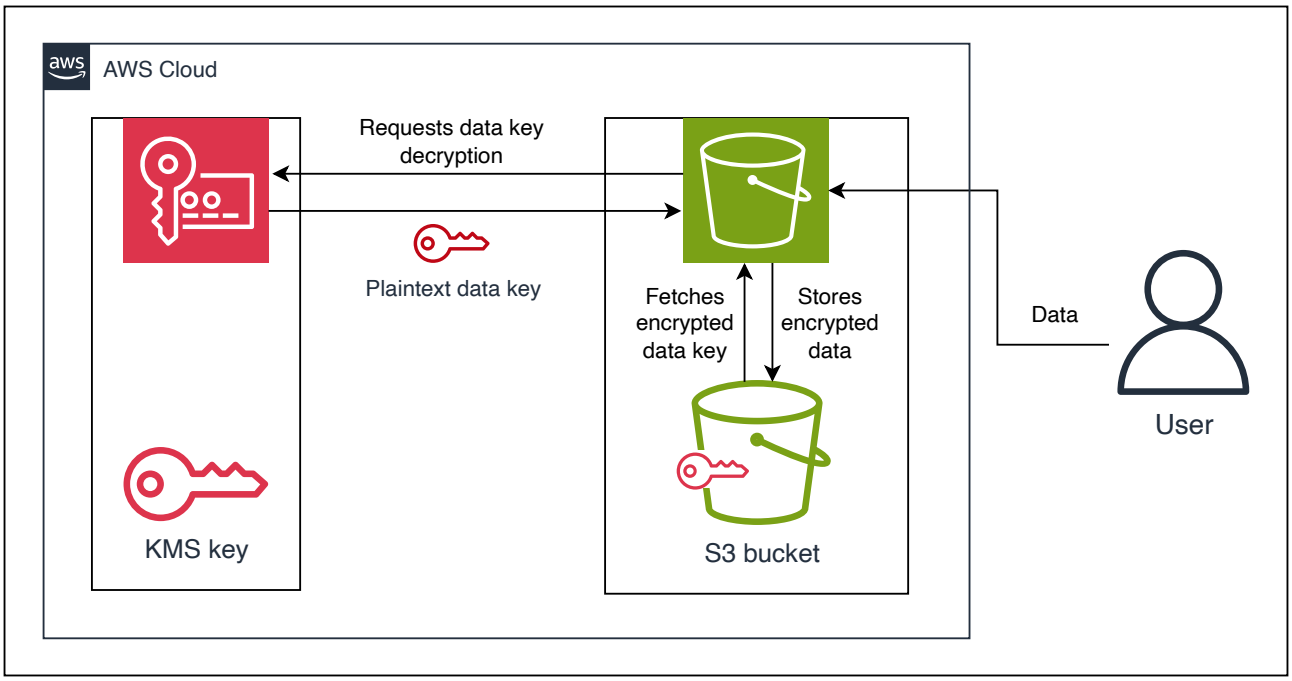
- SSE-S3 (AES-256) allows AWS to handle both the encryption key management and the encryption process. It is the simplest to enable, requiring only a single configuration flag at the bucket or object level. While this method is fully managed and secure, it lacks the detailed audit and access control we get with KMS.
- SSE-KMS integrates Amazon S3 with AWS Key Management Service (KMS). With this method, each object is encrypted with a unique data key, which is protected with a KMS-managed master key (CMK). SSE-KMS allows us to set key usage policies, track access through CloudTrail logs, and control access to object decryption. It's especially useful in regulated environments where data access tracking is critical.

Ask

- SSE-C (server-side encryption with customer-provided keys) allows us to supply our encryption keys when uploading objects to Amazon S3. AWS performs the encryption and decryption using our key, but does not store the key itself. This approach gives us full control over key management, but it also means users are responsible for securely storing and supplying the key with each request. SSE-C is suitable for scenarios where key custody must remain entirely outside AWS, but it comes with added operational complexity and limited support for features like object replication and cross-region access.

For most production-grade applications, especially those requiring fine-grained access control and auditability, SSE-KMS is the go-to choice.

The illustration below shows how encryption works in S3:



S3 bucket data encryption

The shell files below contain the commands to upload a file with SSE-S3 encryption and SSE-KMS:

Ask

SSE...

```
aws s3api put-object \
  --bucket <bucket_name> \
  --key backups/data.csv \
  --body ./data.csv \
  --server-side-encryption AES256
```

Upload object with SSE-S3 encryption

To enforce encryption automatically, we can also enable default encryption at the bucket level. This ensures all new objects written to the bucket are encrypted according to our desired method, either with SSE-S3 or SSE-KMS.

When default encryption is enabled, clients no longer need to specify encryption headers in their API calls. S3 automatically applies the configured encryption scheme to each object. To verify whether default encryption is active, we can retrieve the configuration:

main.sh Shell

```
aws s3api get-bucket-encryption --bucket <bucket_name>
```

Ask

Get configuration

This is particularly valuable in large-scale environments or for teams managing multiple applications writing to a shared bucket, where consistency and policy enforcement are critical.

S3 CORS rules

If the application involves browser-based interactions with S3, such as directly uploading files or loading images into a web UI from an S3 bucket, we'll need to configure CORS (cross-origin resource sharing). CORS rules define which HTTP methods, headers, and origins are permitted to interact with our S3 resources from a different domain.

By default, S3 buckets block all cross-origin requests for security. Without proper CORS configuration, a browser-based app will encounter errors when trying to access S3 resources from a frontend hosted on a separate domain.

A common scenario may involve a React or Angular app hosted on `https://myapp.com` that needs to read from an S3 bucket. In this case, we can explicitly allow that domain and the required HTTP methods.

Here's how we can set up CORS on an S3 bucket, allowing web applications hosted on a different domain (specified by `AllowedOrigins`) to safely access resources (like files) in that S3 bucket using only the `GET` HTTP method.

main.sh Shell

Ask

```
aws s3api put-bucket-cors \
  --bucket <bucket_name> \
  --cors-configuration '
  {
```

```
    "CORSRules": [
      {
        "AllowedOrigins": [https://myapp.com],
        "AllowedMethods": ["GET"],
        "AllowedHeaders": ["*"]
      }
    ]
  }'
```

Set up bucket CORS

We can expand CORS rules to allow multiple origins, methods like `PUT` and `POST` (for uploads), or restrict them to specific headers for greater security.

> **Tip:** Always follow the principle of least privilege to minimize the potential impact of a security breach.

Additionally, we can also inspect the CORS settings as follows:

main.sh Shell

```
aws s3api get-bucket-cors --bucket my-frontend-assets
```

Ask

Inspect CORS settings

S3 MFA delete

When we need an additional layer of protection against accidental or malicious deletions in versioned S3 buckets, MFA delete provides an extra layer of protection. With MFA delete enabled, Amazon S3 requires a second authentication factor, a valid MFA code, for two specific operations: permanent deletion of object versions and disabling versioning.

MFA delete flow

This is especially useful in sensitive environments such as backup archives, compliance-driven data retention, or logs where mistaken deletion can cause significant issues.

> **Note:** MFA delete can only be enabled or disabled using the root user of the AWS account.

Ask

The following commands are used to enable MFA delete and perform MFA delete using AWS CLI commands:

Ena...

```
aws s3api put-bucket-versioning \
  --bucket bucket_name \
  --versioning-configuration Status=Enabled,MFADelete=Enabled \
  --mfa "<MFA_ARN>"
```

Enable MFA delete

MFA delete should be planned carefully and reserved for high-value buckets where immutability is paramount.

S3 access logs

Understanding how our data is being accessed is essential for both security and operational insights. Amazon S3 provides server access logging, a feature that records detailed information about every access request to our bucket, including requester identity, request type, bucket name, operation status, error codes, and timestamps.

These logs are stored in a target S3 bucket that we specify and can be analyzed directly or used as input for monitoring pipelines, billing analysis, or security audits. Common integrations include using Amazon Athena or AWS Glue to query

Ask

access logs using SQL, or forwarding logs to Amazon CloudWatch Logs for visualization and alerting.

Here's how to enable access logging for an S3 bucket `source-data-bucket` to automatically send its access logs to the `access-log-archive` bucket, with the logs being stored under a `logs/` folder within that archive bucket.

index.js C++

```
aws s3api put-bucket-logging \
  --bucket source-data-bucket \
  --bucket-logging-status '
  {
    "LoggingEnabled": {
      "TargetBucket": "access-log-archive",
      "TargetPrefix": "logs/"
    }
  }'
```

Enable AWS access logging

Make sure the target bucket (e.g., `access-log-archive`) grants write permissions to the S3 LogDelivery group. Without this, logs will not be delivered. Here's how to grant access to the target bucket for logging:

index.js C++

```
aws s3api put-bucket-acl \
  --bucket access-log-archive \
  --grant-write 'uri="http://acs.amazonaws.com/groups/s3/LogDeliv
  --grant-read-acp 'uri="http://acs.amazonaws.com/groups/s3/LogDelive
```

Ask

Put bucket access for logging

By analyzing S3 access logs, we gain visibility into who is accessing our data and when, helping detect anomalies, improve performance tuning, and audit for compliance requirements.

Generate and use presigned URLs

Presigned URLs allow developers to generate time-limited, secure access to specific objects in S3 without requiring AWS credentials from the end user. This is particularly useful for scenarios such as securely sharing files, allowing temporary uploads, or enabling limited access to private content, such as images, documents, or video streams.

A presigned URL embeds all necessary permissions to perform the specified operation (like `GET`, `PUT`, or `DELETE`) and expires after a defined interval. This makes it ideal for mobile apps, single-page applications, or temporary third-party access where we don't want to expose S3 buckets or IAM roles directly.

Here's how to generate a presigned URL to download a file `summary.pdf` using AWS CLI:

main.sh Shell

```
aws s3 presign s3://my-private-bucket/reports/summary.pdf \
  --expires-in 3600
```

Ask

Generate presigned URL for downloading the file

This command returns a URL that remains valid for 1 hour (3600 seconds) and can be shared via email or used in client applications to download the object. We can also allow users to upload files directly to S3 without giving them AWS credentials:

main.sh Shell

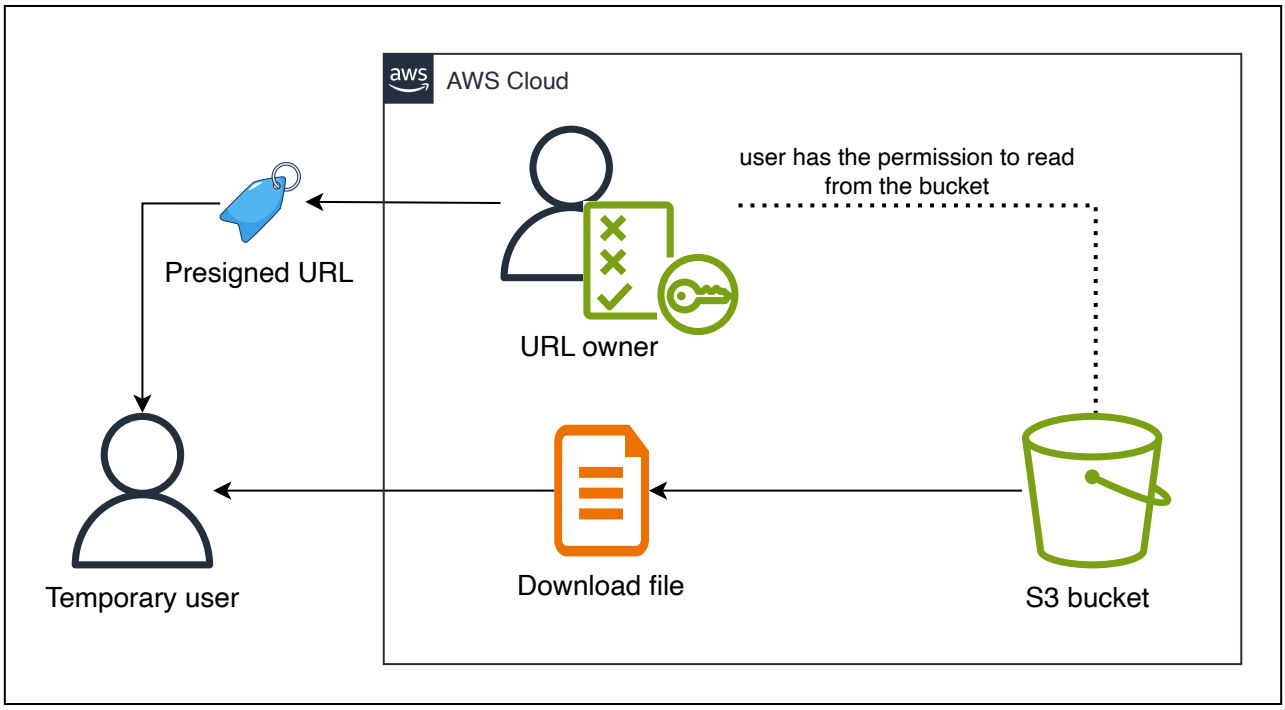
```
aws s3 presign s3://my-private-bucket/uploads/image.jpg \
  --expires-in 600 \
  --http-method PUT
```

Generate presigned URL for uploads

When building with SDKs, generating presigned URLs is supported in all m languages, including Boto3 (Python), AWS SDK for JavaScript, and Go. The

Ask

illustration below shows how we can share and upload objects using presigned URLs:



1 / 2

Share objects using the presigned URL

Presigned URLs are a best practice for securely delegating limited access to objects, enforcing time boundaries, and reducing backend load by offloading direct file transfers to S3.

Conclusion

Amazon S3 offers a comprehensive set of tools to help developers secure data, enforce access policies, and maintain compliance. Whether we are encrypting data, configuring cross-origin access, or granting temporary access via pr URLs, these features allow fine-grained control over how and when S3 objects are

Ask